

UF7

Servicios web

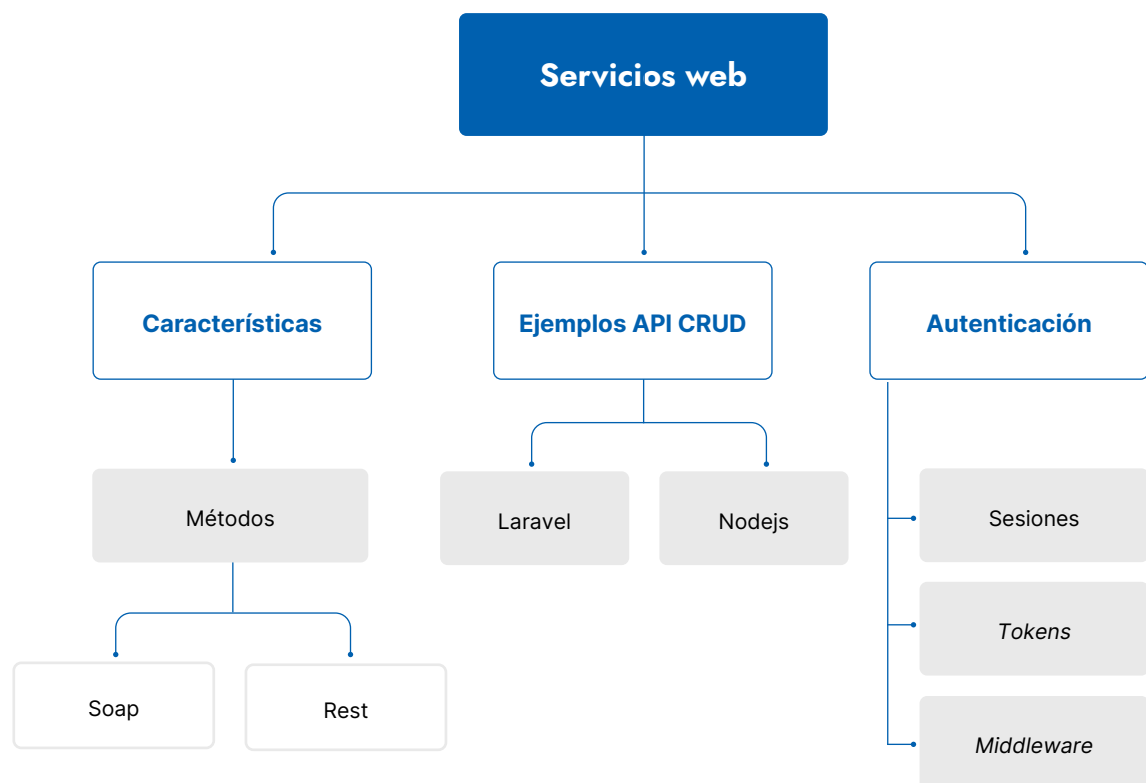
Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	04
1. Inyección de dependencias	05
1.1. ¿Qué es un servicio web?	05
1.2. La tecnología que hace posible un servicio web, con un ejemplo ...	06
1.3. Ventajas y desventajas de los servicios web.....	07
1.4. Qué es SOAP	07
1.5. Qué es REST	09
1.6. Conclusiones: SOAP frente a REST	10
2. Servicios web API CRUD con PHP	12
2.1. El protocolo HTTP	12
2.2. Definir los controladores de API.....	14
A. Establecer las rutas	15
B. Servicios GET	15
C. Resto de servicios	16
D. Validación de datos	17
E. Respuestas de error.....	17
2.3. Probar los servicios con Postman.....	18
2.4. Añadir otros tipos de peticiones (POST, PUT, DELETE)	18
3. Servicios web API CRUD con NodeJS	19
3.1. ¿Qué es Express?	19
A. Descarga e instalación	19
B. Ejemplo de servidor básico con Express.....	20
C. Elementos básicos: aplicación, petición y respuesta.....	21

3.2. Ejemplo de enrutamiento simple	22
A. Listado de todos los contactos	24
B. Ficha de un contacto a partir de su id	24
C. Inserción de contactos (POST)	25
D. Modificación de contactos (PUT)	26
4. Mecanismos de autenticación y verificación	27
4.1. Fundamentos de la autenticación basada en <i>tokens</i>	27
4.2. Implementación de la autenticación basada en <i>tokens</i> con Laravel	28
A. Preparar el ejemplo base	29
4.3. Autenticación basada en <i>tokens</i> nativa con Laravel	33
A. Configuración básica	33
B. Protección de rutas	34

Mapa conceptual



01 Inyección de dependencias

Hoy en día, usar diferentes servicios a través de la web es una actividad habitual. Comprar on-line, leer el periódico, reservar una mesa en un restaurante o ver películas son solo algunos ejemplos de las muchas interacciones que se producen a diario entre usuarios y máquinas. Pero, además, y aunque pueda pasar desapercibido, estas interacciones también tienen lugar entre máquinas: el cliente y el servidor se están enviando continuamente solicitudes y respuestas, transmisión que se produce gracias a los *web services* o servicios web.

1.1. ¿Qué es un servicio web?

Un *web service* facilita un **servicio a través de Internet**: se trata de una interfaz mediante la que dos máquinas (o aplicaciones) se comunican entre sí. Esta tecnología se caracteriza por estos dos rasgos:

Multiplataforma: cliente y servidor no tienen por qué contar con la misma configuración para comunicarse. El servicio web se encarga de hacerlo posible.

Distribuida: por lo general, un servicio web no está disponible para un único cliente, sino que son diferentes los que acceden a él a través de Internet.

Cuando se utiliza un servicio web, un cliente manda una solicitud a un servidor, y desencadena una acción por parte de este. A continuación, el servidor devuelve una respuesta al cliente.

Características

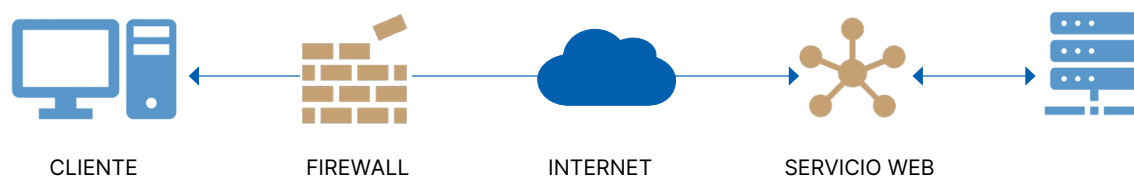
Existen numerosos protocolos que permiten la comunicación entre ordenadores a través de una red: FTP, HTTP, SMTP, POP3, Telnet, etc. En todos estos protocolos se definen un servidor y un cliente. El servidor es la máquina que está esperando conexiones (escuchando) por parte de un cliente. El cliente es la máquina que inicia la comunicación. Cada uno de estos protocolos tiene asignado además un puerto concreto (TCP o UDP), que será el que utilicen normalmente los equipos servidores.

Cada uno de los protocolos que hemos nombrado ha sido creado para un fin específico: FTP, para transferencia de archivos; HTTP, para páginas web; SMTP y POP3, para correo electrónico, y Telnet, para acceso remoto. No han sido diseñados para transportar peticiones de información genéricas entre aplicaciones, como solicitar el PVP de un producto. Sin embargo, desde hace ya tiempo existen otras soluciones para este tipo de problemas. Una de las más populares es RPC.

El protocolo RPC se creó para permitir a un sistema acceder de forma remota a funciones o procedimientos que se encuentren en otro sistema. El cliente se conecta con el servidor, y le indica qué función debe ejecutar. El servidor la ejecuta y le devuelve el resultado obtenido. Así, por ejemplo, podemos crear en el servidor RPC una función que reciba un código de producto y devuelva su PVP.

RPC usa su propio puerto, pero normalmente solo a modo de directorio. Los clientes se conectan a él para obtener el puerto real del servicio que les interesa. Este puerto no es fijo; se asigna de forma dinámica.

Los servicios web se crearon para permitir el intercambio de información al igual que RPC, pero sobre la base del protocolo HTTP (de ahí el término «web»). En lugar de definir su propio protocolo para transportar las peticiones de información, utilizan HTTP para este fin. La respuesta obtenida no será una página web, sino la información que se solicitó. De esta forma, pueden funcionar sobre cualquier servidor web, y, lo que es aún más importante, utilizando el puerto 80, reservado para este protocolo. Por tanto, cualquier ordenador que pueda consultar una página web podrá también solicitar información de un servicio web. Si existe algún cortafuegos en la red, tratará la petición de información igual que lo haría con la solicitud de una página web.



Esquema 1.

IMPORTANTE

Al principio, *web service* solo funcionaba con SOAP, pero en la actualidad se utilizan también otros métodos como REST.

1.2. La tecnología que hace posible un servicio web, con un ejemplo

IMPORTANTE

Todos los *web services* cuentan con un *uniform resource identifier (URI)* unívoco, esto es, la dirección del servicio web. Es similar al *uniform resource locator (URL)* que permite acceder a páginas web. El **catálogo UDDI** (*universal description, discovery and integration*) era un registro en XML del catálogo de todos los servicios web; debía desempeñar también un papel importante, pues permitía encontrar los servicios web, pero este servicio nunca logró imponerse y sus mayores partidarios terminaron retirándose del proyecto.

Más importancia tiene el lenguaje *web service description language (WSDL)*. Un servicio web contiene un **archivo en WSDL** en el que se describe el servicio de forma detallada.

Con esta información, el cliente puede comprender qué funciones puede ejecutar en el servidor a través del servicio web. La comunicación funciona exclusivamente mediante diferentes protocolos y arquitecturas. Entre ellos, son muy populares el protocolo de red SOAP, en combinación con el estándar de Internet HTTP o los servicios web basados en una arquitectura REST.

Con estas tecnologías se posibilita el intercambio de peticiones y respuestas a menudo utilizando **el lenguaje de marcado extensible (XML)**. Este lenguaje, relativamente simple, puede ser interpretado en igual medida por personas y ordenadores, y además es adecuado para unir sistemas con requisitos diferentes. Con todo, REST también admite otros formatos, como JSON.

Veamos cómo funciona la mecánica de esta tecnología con un ejemplo de *web service*. Partamos de un software escrito en Visual Basic que se ejecuta en una máquina con sistema Windows. El programa necesita el servicio de un servidor web Apache. Para ello, el cliente envía una **solicitud SOAP en forma de mensaje HTTP** al servidor. El *web service* interpreta el contenido de la solicitud y se encarga de que el servidor lleve a cabo una acción. Finalmente, el servicio web formula una respuesta y la envía de vuelta al cliente (de nuevo con SOAP y HTTP), que vuelve a interpretarla. La información se envía entonces al software, donde será procesada.

1.3. Ventajas y desventajas de los servicios web

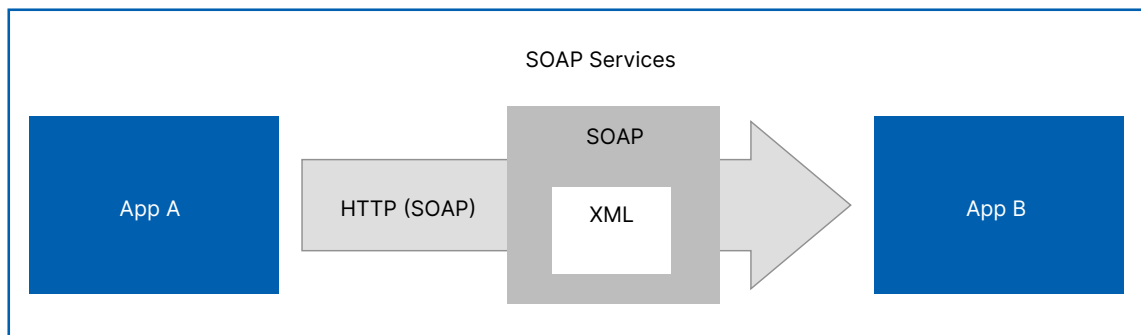
La ventaja principal de los servicios web es que la comunicación no depende de una plataforma determinada, por lo que el cliente y el servidor apenas han de presentar rasgos en común para poder comunicarse. Para ello, la tecnología *web service* recurre a formatos estandarizados que interpretan todos los sistemas.

Pero en estos formatos es donde encontramos una de las desventajas. Precisamente, XML es un formato más bien voluminoso, que genera grandes paquetes de datos, lo que puede crear problemas en las conexiones de red lentas. Otra posibilidad que permite conectar a dos sistemas a través de Internet son las API web. Aunque, por lo general, son más rápidas, someten a cliente y servidor a especificaciones más concretas, con lo que la interoperabilidad se ve limitada.

1.4. Qué es SOAP

Los servicios SOAP, mejor conocidos simplemente como *web services*, se vienen utilizando desde los años noventa; son servicios que basan su comunicación en el protocolo SOAP (*simple object access protocol*), al cual se refiere Wikipedia como «protocolo estándar

que define cómo dos objetos en diferentes procesos pueden comunicarse por medio de intercambio de datos XML». Por lo tanto, queda claro que la comunicación se realiza mediante XML, lo cual nos debe quedar muy claro, pues es en este aspecto donde radican las principales diferencias con REST. Los servicios SOAP funcionan por lo general por el protocolo HTTP, que es lo más habitual cuando invocamos un *web service*; sin embargo, SOAP no está limitado a este protocolo, si no que puede ser enviado por FTP, POP3, TCP o colas de mensajería (JMS, MQ, etc.). Pero, como comentábamos, HTTP es el protocolo principal.



Esquema 2.

SOAP es un formato más pesado, tanto en tamaño como en procesamiento, pues los XML tienen que ser parseados a un árbol DOM y resolver espacios de nombre (**namespaces**) antes de poder empezar a procesar el documento. Los XML, además, tienen métodos de validación muy potentes y ampliamente utilizados, a diferencia de JSON, que sí tiene forma de validar.

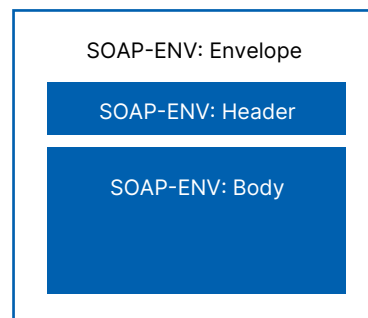
SOAP solo admite formato XML, por lo que, cuando lo que necesitamos es flexibilidad y un rendimiento superior, podemos optar por REST.

Una gran diversidad de servicios web se basa en SOAP. Por ejemplo, **Amazon e eBay** trabajan (parcialmente) con el protocolo de red.

La estructura de SOAP: funcionamiento del protocolo

SOAP se basa en el **metalenguaje XML**. Este, que también es una recomendación de W3C, es un conjunto de unidades de información que son necesarias para dar como resultado un documento XML bien formado (es decir, de acuerdo con una estructura recomendada). SOAP asimila su estructura de mensajes y por consiguiente se corresponde principalmente con un documento XML.

El transporte se realiza a través del protocolo y se integra en su estructura. Un mensaje HTTP con una solicitud SOAP tiene la siguiente forma:



Esquema 3.

En este ejemplo, la solicitud comienza con un encabezado HTTP. A continuación, sigue el llamado **envelope (sobre) de SOAP**, que envuelve el contenido real del mensaje como un sobre. Los elementos principales de SOAP son el *header* (encabezado) y el *body* (cuerpo).

» **Header:** el encabezado de la solicitud SOAP contiene metadatos, como la encriptación que se ha utilizado. Su uso es opcional.

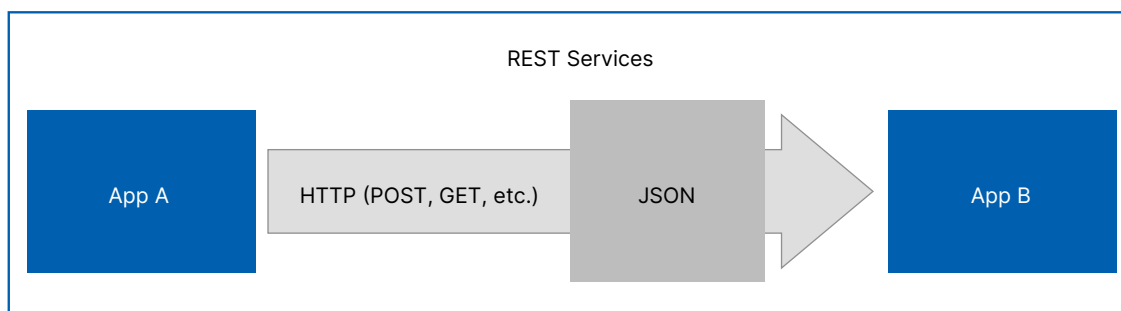
» **Body:** en el cuerpo del mensaje se encuentran los datos en sí.

Los conceptos utilizados en el *body* no tienen nada que ver con SOAP, sino que dependen completamente de la aplicación.

El protocolo aparece a menudo en combinación con el lenguaje WSDL (*web services description language*). Se trata de un **lenguaje de descripción** especial para servicios web que, por otra parte, no depende de la plataforma. Con su ayuda, un cliente puede reconocer qué servicios finales (*endpoints*) ofrece una API SOAP y cómo interactuar con ellos (parámetros, codificación, operaciones...). A partir del archivo WSDL, el cliente deduce qué posibilidades tiene para realizar una solicitud SOAP. El dúo WSDL y SOAP permite que dos sistemas diferentes se comuniquen sin tener que adaptarse previamente.

1.5. Qué es REST

Por otra parte, tenemos REST; es una tecnología mucho más flexible y nueva, que transporta datos por medio del protocolo HTTP, pero este permite utilizar los diversos métodos que proporciona HTTP para comunicarse (como son GET, POST, PUT, DELETE o PATCH), y a la vez utiliza los códigos de respuesta nativos de HTTP (404, 200, 204, 409). REST es tan flexible que permite transmitir prácticamente **cualquier tipo de datos**, ya que el tipo de datos está definido por el *header content-type*, lo que nos permite mandar XML, JSON, binarios (imágenes, documentos), Text, etc., lo que contrasta con SOAP, que solo permite la transmisión de datos en formato XML. A pesar de la gran variedad de tipos de datos que podemos mandar con REST, la gran mayoría transmite en JSON, por un motivo muy importante: **JSON es interpretado de forma natural por JavaScript**, lo que ha hecho que *frameworks* como Angular y React se aprovechen al máximo, pues pueden enviar peticiones directas al servidor por medio de AJAX y obtener los datos de una forma nativa. Los formularios de HTML pueden ser apuntados a los servicios REST sin ningún problema (por ejemplo).



Esquema 4.

1.6. Conclusiones: SOAP frente a REST

Tanto SOAP como REST (que, al fin y al cabo, es una arquitectura y no un protocolo) se utilizan para servicios web, aunque procesan las tareas de manera distinta. Mientras que SOAP es algo más antiguo, REST (también conocido como *RESTful web services*) ha ido ganando terreno y en la actualidad distribuye **aproximadamente el 70 % de los servicios web**.

No obstante, cada uno ofrece ventajas diferentes: REST está considerado relativamente sencillo, no trabaja solo con XML, es más rápido y, en comparación con SOAP, es más ligero. La libertad que caracteriza a REST a la hora de elegir si trabajar con XML (a menudo recurre a JSON) caracteriza a SOAP a la hora de elegir el protocolo. Aunque HTTP es a menudo la opción elegida, teóricamente SOAP funciona también en combinación con **FTP, SMTP u otros protocolos**.

Respecto a la seguridad, SOAP supone una gran ventaja: **WS-Security** (especificaciones para los criterios de seguridad respecto a servicios web) está anclado en el protocolo de red. También el tratamiento de los errores está mejor regulado en SOAP, porque incorpora directamente una función para la repetición de la solicitud.

IMPORTANTE

Tanto SOAP como REST presentan ventajas y desventajas, de modo que no se puede concluir que una solución sea mejor que la otra. Sin embargo, por su sencillez, la mayoría de los desarrolladores recurren a REST. Finalmente, la decisión depende del lenguaje de programación que se utilice y del contexto en el que se quiera utilizar el protocolo.

Profundizando más en REST, las estrictas normas de la arquitectura REST hacen posible el desarrollo de servicios bien estructurados, fáciles de integrar y que se **comunican por medio de un protocolo normalizado (HTTP)**. Gracias a una estructura orientada a los recursos, en la concepción de un REST *web service* o servicio web REST la búsqueda de protocolos específicos para aplicaciones no es necesaria, al contrario que en el caso de alternativas como SOAP.

Además, el estándar HTTP define una **gama completa de comandos** por medio de los que se puede acceder a los recursos:

MÉTODO HTTP (COMANDO)	DESCRIPCIÓN
GET	Solicita el recurso correspondiente al servidor sin modificar su estado.
POST	Crea un nuevo recurso subyacente al recurso facilitado y el URI lo dirige automáticamente. También puede usarse para modificar recursos ya existentes.
HEAD	Solo solicita el encabezado del recurso al servidor para comprobar, por ejemplo, la validez de un archivo.
PUT	Coloca el recurso solicitado en el servidor o modifica uno ya existente.
PATCH	Modifica una parte del recurso facilitado.
TRACE	Devuelve la solicitud tal y como la recibe el servidor web para determinar los cambios en el camino al servidor.
OPTIONS	Muestra una lista de los métodos soportados por el servidor.
CONNECT	Tramita la solicitud a través de un túnel SSL para, por lo general, establecer una conexión por medio de un servidor <i>proxy</i> .

Tabla 1.

La estructura REST ofrece medios excelentes para la concepción e implementación de todo tipo de servicios web. Gracias a que casi todos los dispositivos admiten HTTP, tanto los **clientes móviles como los de escritorio** pueden trabajar con la interfaz de REST con facilidad y sin necesidad de realizar implementaciones adicionales. El resultado son servicios web que sorprenden gracias al alto grado de los siguientes aspectos:

- » independencia de plataforma;
- » escalabilidad;
- » rendimiento;
- » interoperabilidad, y
- » flexibilidad.

Al principio, *web service* solo funcionaba con SOAP, pero en la actualidad el mayoritariamente utilizado es REST.

02 Servicios web API CRUD con PHP

En este apartado vamos a construir con un ejemplo un servicio web de CRUD en lenguaje PHP. Vamos a ver qué pasos dar para construir una API REST básica en Laravel que dé soporte a las operaciones básicas sobre una o varias entidades: consultas (GET), inserciones (POST), modificaciones (PUT) y borrados (DELETE). Emplearemos para ello los denominados controladores de API, que comentamos brevemente en la Unidad 5, y que proporcionan un conjunto de funciones ya definidas para dar soporte a cada uno de estos comandos.

A estas alturas, todos deberíamos tener claro que cualquier aplicación web se basa en una arquitectura cliente-servidor, donde un servidor queda a la espera de conexiones de clientes, y los clientes se conectan a los servidores para solicitar ciertos recursos. Sobre esta base, veremos unas breves pinceladas de cómo funciona el protocolo HTTP, y en qué consisten los servicios REST de manera práctica.

2.1. El protocolo HTTP

Las comunicaciones web entre cliente y servidor se realizan mediante el protocolo HTTP (o HTTPS, en el caso de comunicaciones seguras). En ambos casos, cliente y servidor se envían cierta información estándar, en cada mensaje.

En cuanto a los clientes, envían al servidor los datos del recurso que solicitan, junto con cierta información adicional, como, por ejemplo, las cabeceras de petición (información relativa al tipo de cliente o navegador, contenido que acepta, etc.), y parámetros adicionales llamados normalmente datos del formulario, puesto que suelen contener la información de algún formulario que se envía de cliente a servidor.

Por lo que respecta a los servidores, aceptan estas peticiones, las procesan y envían de vuelta algunos datos relevantes, como un código de estado (que indica si la petición pudo ser atendida satisfactoriamente o no), cabeceras de respuesta (que indican el tipo de contenido enviado, tamaño, idioma, etc.), y el recurso solicitado propiamente dicho, si todo ha ido correctamente.

Este es el mecanismo que hemos estado utilizando hasta ahora a través de los controladores: reciben la petición concreta del cliente y envían una respuesta, que por el momento se ha centrado en renderizar un contenido HTML de una vista.

En cuanto a los códigos de estado de la respuesta, dependen del resultado de la operación que se haya realizado; se catalogan en cinco grupos:

Códigos 1XX	Representan información sobre una petición normalmente incompleta. No son muy habituales, pero se pueden emplear cuando la petición es muy larga, y se envía antes una cabecera para comprobar si se puede procesar dicha petición.
Códigos 2XX	Representan peticiones que se han podido atender satisfactoriamente. El código más habitual es el 200, respuesta estándar para las peticiones que son correctas. Existen otras variantes, como el código 201, que se envía cuando se ha insertado o creado un nuevo recurso en el servidor (una inserción en una base de datos, por ejemplo), o el código 204, que indica que la petición se ha atendido bien, pero no se ha devuelto nada como respuesta.
Códigos 3XX	Son códigos de redirección, que indican que, de algún modo, la petición original se ha redirigido a otro recurso del servidor. Por ejemplo, el código 301 indica que el recurso solicitado se ha movido permanentemente a otra URL. El código 304 indica que el recurso solicitado no ha cambiado desde la última vez que se solicitó, por si se quiere recuperar de la caché local en ese caso.
Códigos 4XX	Indican un error por parte del cliente. El más típico es el error 404, que indica que estamos solicitando una URL o recurso que no existe. Pero también hay otros habituales, como el 401 (cliente no autorizado), o el 400 (los datos de la petición no son correctos; por ejemplo, porque los campos del formulario no son válidos).
Códigos 5XX	Indican un error por parte del servidor. Por ejemplo, el error 500 indica un error interno del servidor, o el 504 es un error de <i>timeout</i> , por tiempo excesivo en emitir la respuesta.

Haremos uso de estos códigos de estado en nuestros servicios REST para informar al cliente del tipo de error que se haya producido, o del estado en que se ha podido atender su petición.

API (*application programming interface*) es básicamente el conjunto de métodos o funcionalidades que se ponen a disposición de quienes los quieran utilizar. En este caso, el concepto de API REST hace referencia al conjunto de servicios REST proporcionados por el servidor para los clientes que quieran utilizarlos.

2.2. Definir los controladores de API

Para proporcionar una API REST a los clientes que lo requieran, necesitamos definir un controlador o unos controladores orientados a ofrecer estos servicios REST. Estos controladores en Laravel se denominan de tipo API, como vimos en unidades previas. Normalmente, se definirá un controlador API por cada uno de los modelos a los que necesitemos acceder. Vamos a crear uno de prueba para ofrecer una API REST sobre los libros de nuestra aplicación de biblioteca.

Existen diferentes formas de ejecutar el comando de creación del controlador de API. Aquí vamos a mostrar quizá una de las más útiles:

```
php artisan make:controller Api/LibroController --api --model=Libro
```

Esto creará el controlador en la carpeta `App\Http\Controllers\Api` con una serie de funciones ya predefinidas. No es obligatorio ubicarlo en esa subcarpeta, obviamente, pero esto nos servirá para separar los controladores API del resto. Observemos que se incorpora automáticamente la cláusula `use` para cargar el modelo asociado, que hemos indicado en el parámetro `--model`. Además, los métodos `show`, `update` y `destroy` ya vienen con un parámetro de tipo `Libro`, que facilitará mucho algunas tareas.

Cada una de las funciones del nuevo controlador creado se asocia a uno de los métodos REST comentados anteriormente:

MÉTODO (CONTROLADOR)	DESCRIPCIÓN
<code>index</code>	Se asociaría con una operación GET de listado general, para obtener todos los registros (de libros, en este caso).
<code>store</code>	Se asociaría con una operación POST, para almacenar los datos que lleguen en la petición (como un nuevo libro, en nuestro caso).
<code>show</code>	Se asociaría con una operación GET para obtener el registro asociado a un identificador concreto.
<code>update</code>	Se asociaría con una operación PUT, para actualizar los datos del registro asociado a un identificador concreto.
<code>destroy</code>	Se asociaría con una operación DELETE, para eliminar los datos del registro asociado a un identificador concreto.

Tabla 2.

A. Establecer las rutas

Una vez que tenemos el controlador API creado, vamos a definir las rutas asociadas a cada método del controlador. Como vimos en unidades anteriores, podíamos emplear el método `Route::resource` en el archivo `routes/web.php` para establecer de golpe todas las rutas asociadas a un controlador de recursos. De forma análoga, podemos emplear el método `Route::apiResource` en el archivo `routes/api.php` para establecer automáticamente todas las rutas de un controlador de API. Añadimos esta línea en dicho archivo `routes/api.php`:

Desde la versión 8.x en adelante hay que ponerlo de esta forma y no olvidar usar la directiva `use` para importar la clase.

```
Route::resource('libros', LibroController::class);
```

Las rutas de API (aquellas definidas en el archivo `routes/api.php`) tienen, de forma predeterminada, un prefijo `api`, tal y como se establece en el provider `RouteServiceProvider`. Por tanto, hemos definido una ruta general `api/libros`, de forma que todas las subrutas que se deriven de ella llevarán a uno u otro método del controlador de API de libros.

Podemos comprobar qué rutas hay activas con este comando:

```
php artisan route:list
```

Si ejecutamos, veremos las siguientes rutas:

```
GET|HEAD api/libros ..... libros.index > Api\librosController@index
POST api/libros ..... libros.store > Api\librosController@store
GET|HEAD api/libros/{libro} ..... libros.show > Api\librosController@show
PUT|PATCH api/libros/{libro} ..... libros.update > Api\librosController@update
DELETE api/libros/{libro} ..... libros.destroy > Api\librosController@destroy
GET|HEAD api/user .....
GET|HEAD libros ..... libros.index > libroController@index
POST libros ..... libros.store > libroController@store
GET|HEAD libros/crear ..... libros.create > libroController@create
GET|HEAD libros/{libro} ..... libros.show > libroController@show
PUT|PATCH libros/{libro} ..... libros.update > libroController@update
DELETE libros/{libro} ..... libros.destroy > libroController@destroy
GET|HEAD libros/{libro}/editar ..... libros.edit > libroController@edit
GET|HEAD login ..... login > loginController@login
POST login ..... login > loginController@login
```

Figura 1.

B. Servicios GET

Vamos a empezar por definir el método `index`. En este caso, vamos a obtener el conjunto de libros de la base de datos y devolverlo tal cual:

```
public function index()
{
    $libros = Libro::all();
    return response()->json($libros, 200);
}
```

C. Resto de servicios

Veamos ahora cómo implementar el resto de servicios (POST, PUT y DELETE). En el caso de la inserción (POST), deberemos recibir en la petición los datos del objeto que se va a insertar (un libro, en nuestro ejemplo). Igual que los datos del servidor al cliente se envían en formato JSON, es de esperar en aplicaciones que siguen la arquitectura REST que los datos del cliente al servidor también se envíen en formato JSON.

Nuestro método **store**, asociado al servicio POST, podría quedar de este modo (devolvemos el código de estado 201, que se utiliza cuando se han insertado elementos nuevos):

```
public function store(LibroPost $request)
{
    $libro = new Libro();
    $libro->titulo = $request->titulo;
    $libro->editorial = $request->editorial;
    $libro->precio = $request->precio;
    $libro->autor()->associate(Autor::findOrFail($request->autor_id));
    $libro->save();

    return response()->json($libro, 201);
}
```

De forma similar, implementaríamos el servicio PUT, a través del método **update**. En este caso, devolvemos un código de estado 200:

```
public function update(LibroPost $request, Libro $libro)
{
    $libro->titulo = $request->titulo;
    $libro->editorial = $request->editorial;
    $libro->precio = $request->precio;
    $libro->autor()->associate(Autor::findOrFail($request->autor_id));
    $libro->save();

    return response()->json($libro, 200);
}
```

Finalmente, para el servicio DELETE, debemos implementar el método **destroy**. Es habitual en este tipo de operaciones de borrado devolver en formato JSON el objeto que se ha eliminado, por si acaso se quiere deshacer la operación en un paso posterior. En este caso, el código del método de borrado sería así:


```
public function destroy(Libro $libro)
{
    $libro->delete();
    return response()->json($libro, 200);
}
```

Como podemos empezar a intuir, probar estos servicios no es tan sencillo como probar servicios de tipo GET, ya que no podemos simplemente teclear una URL en el navegador. Necesitamos un mecanismo para pasarle los datos al servidor en formato JSON, y también el método (POST, PUT o DELETE). Veremos cómo con la herramienta **POSTMAN**.

D. Validación de datos

A la hora de recibir datos en formato JSON para servicios REST, también podemos establecer mecanismos de validación similares a los vistos para los formularios, a través de los correspondientes requests. De hecho, en el caso de la biblioteca podemos emplear la clase `App\Http\Requests\LibroPost`, que hicimos en unidades anteriores, para validar que los datos que llegan tanto a `store` como a `update` son correctos. Basta con usar un parámetro de este tipo en estos métodos, en lugar del parámetro `Request`, que viene por defecto:

```
public function store(LibroPost $request)
{
    ...
}

public function update(LibroPost $request, Libro $libro)
{
    ...
}
```

E. Respuestas de error

Por otra parte, debemos asegurarnos de que cualquier error que se produzca en la parte de la API devuelva un contenido en formato JSON, y no una página web. Por ejemplo, si solicitamos ver la ficha de un libro cuyo `id` no existe, no debería devolvernos una página de error 404, sino un código de estado 404 con un mensaje de error en formato JSON.

2.3. Probar los servicios con Postman

Ya hemos visto que probar unos servicios de listado (GET) es sencillo a través de un navegador. Pero los servicios de inserción (POST), modificación (PUT) o borrado (DELETE) exigen de otras herramientas para poder ser probados. Podríamos definir formularios con estos métodos encapsulados, pero el esfuerzo de definir esos formularios para luego no utilizarlos más no merece mucho la pena. Veremos a continuación una herramienta muy útil para probar todo tipo de servicios sin necesidad de implementar nada adicional.

Postman es una aplicación gratuita y multiplataforma que permite enviar todo tipo de peticiones a un servidor determinado, y examinar la respuesta que este produce. De esta forma, podemos comprobar que los servicios ofrecen la información adecuada antes de ser usados por una aplicación cliente real.

Para descargar e instalar Postman, debemos ir a su web oficial <https://www.postman.com/>, a la sección de descargas, y descargar la aplicación. Es un archivo portable, que se descomprime y dentro está la aplicación lista para ejecutarse.

2.4. Añadir otros tipos de peticiones (POST, PUT, DELETE)

Las peticiones POST difieren de las peticiones GET en que se envía cierta información en el cuerpo de la petición. Esta información normalmente son los datos que se quieren añadir en el servidor. ¿Cómo podemos hacer esto con Postman?

En primer lugar, creamos una nueva petición, elegimos el comando POST y definimos la URL (en nuestro caso, `localhost:8000/api/libros` o algo similar, dependiendo de cómo tengamos en marcha el servidor). Entonces, hacemos clic en la pestaña Body, bajo la URL, y establecemos el tipo como raw para que nos deje escribirlo sin restricciones. También conviene cambiar la propiedad Text para que sea JSON, y que así el servidor recoja el tipo de dato adecuado. Se añadirá automáticamente una cabecera de petición (header) que especificará que el tipo de contenido que se va a enviar son datos JSON. Después, en el cuadro de texto situado bajo estas opciones, especificamos el objeto JSON que queremos enviar para insertar.

03 Servicios web API CRUD con NodeJS

En este apartado vamos a construir con un ejemplo un servicio web de CRUD en lenguaje NodeJS. Vamos a ver qué pasos dar para construir una API REST básica en NodeJS que dé soporte a las operaciones básicas sobre una o varias entidades: consultas (GET), inserciones (POST), modificaciones (PUT) y borrados (DELETE). Emplearemos para ello los denominados controladores de API, que proporcionan un conjunto de funciones ya definidas para dar soporte a cada uno de estos comandos.

En esta sesión del tema veremos cómo aplicar lo aprendido hasta ahora para desarrollar un servidor sencillo que proporcione una API REST a los clientes que se conecten en el lenguaje de servidor NodeJS. Por lo tanto, identificando el recurso que se va a solicitar y el comando que se le va a aplicar, el servidor que ofrece esta API REST proporciona una respuesta a esa petición. Esta respuesta viene dada típicamente por un mensaje en formato JSON o XML (aunque este cada vez está más en desuso).

3.1. ¿Qué es Express?

Express es un *framework* ligero y, a la vez, flexible y potente para desarrollo de aplicaciones web con Node. En primer lugar, se trata de un *framework* ligero porque no viene cargado de serie con toda la funcionalidad que un *framework* podría tener, a diferencia de otros *frameworks* más pesados y autocontenidos como Symfony o Ruby on Rails. Pero, además, se trata de un *framework* flexible y potente porque permite añadirle, a través de módulos Node y de *middleware*, toda la funcionalidad que se requiera para cada tipo de aplicación. De este modo, podemos utilizarlo en su versión más ligera para aplicaciones web sencillas, y dotarlo de más elementos para desarrollar aplicaciones muy complejas.

Como veremos, con Express podemos desarrollar tanto servidores típicos de contenido estático (HTML, CSS y Javascript) como servicios web accesibles desde un cliente, y, por supuesto, aplicaciones que combinen ambas cosas.

A. Descarga e instalación

La instalación de Express es tan sencilla como la de cualquier otro módulo que queramos incorporar a un proyecto Node. Simplemente necesitamos ejecutar el correspondiente comando `npm install` (y, previamente, el comando `npm init`, en el caso de que aún no hayamos creado el archivo `package.json`):

```
npm install express
```

B. Ejemplo de servidor básico con Express

Crea un proyecto llamado **PruebaExpress** en tu carpeta **ProyectosNode/Pruebas**, e instala Express en él. Después, crea un archivo llamado **index.js** y deja en él este código:

```
const express = require('express');  
let app = express();  
app.listen(8080);
```

El código, como ves, es muy sencillo. Primero incluimos la librería, después inicializamos una instancia de Express (normalmente se almacena en una variable llamada **app**), y, finalmente, la ponemos a escuchar por el puerto que queramos. En este caso, se ha escogido el puerto 8080, para no interferir con el puerto predeterminado por el que se escuchan las peticiones HTTP, que es el 80.

Para poner en marcha el servidor es muy fácil; basta con abrir un navegador y acceder a la URL

http://localhost:8080

Pero con cada cambio que hagamos en el programa tenemos que parar el servidor y volver a iniciarlo. Para ello existe una utilidad llamada **nodemon**, la cual permite evitarnos tener que reiniciar el servidor. Podemos instalar **nodemon** globalmente con **npm**:

```
npm install nodemon -g
```

Desde la consola de nuestro IDE podemos ejecutar

```
$ nodemon index.js
```

Cada vez que realicemos un cambio en un archivo con una de las extensiones vigiladas de forma predeterminada (.js, .mjs, .json) en el directorio actual o en un subdirectorio, el proceso se reiniciará automáticamente.

Si pruebas a ejecutar la aplicación, verás que no finaliza. Hemos creado un pequeño servidor Express que se queda a la espera de recibir peticiones.

Partiendo de la base anterior, vamos a añadir una serie de rutas en nuestro servidor principal para dar soporte a los servicios asociados a ellas. Una vez que hemos inicializado la aplicación (variable **app**), basta con ir añadiendo métodos (**get**, **post**, **put** o **delete**), indicando para cada uno la ruta que debe atender, y el callback o función que se ejecutará en ese caso. Por ejemplo, para atender a una ruta llamada **/bienvenida** por GET, añadiríamos este método:

```
let app = express();  
app.get('/bienvenida', (req, res) => {  
  res.send('Hola, bienvenido/a');  
});
```

El callback en cuestión recibe dos parámetros siempre: el objeto que contiene la petición (típicamente llamado **req**, abreviatura de request), y el objeto para emitir la respuesta (típicamente llamado **res**, abreviatura de response). Más adelante, veremos qué otras cosas podemos hacer con estos objetos, pero de momento emplearemos la respuesta para enviar (**send**) texto al cliente que solicitó el servicio, y **req** para obtener determinados datos de la petición.

Podemos volver a lanzar el servidor Express, y probar este nuevo servicio accediendo a la URL correspondiente:

http://localhost:8080/bienvenida

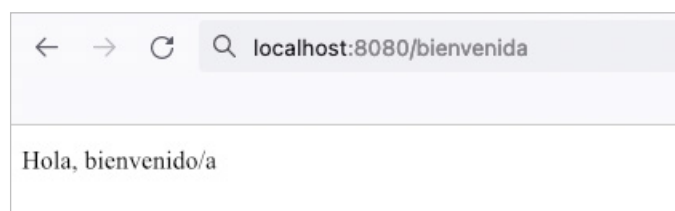


Figura 3.

C. Elementos básicos: aplicación, petición y respuesta

Existen tres elementos básicos sobre los que se sustenta el desarrollo de aplicaciones en Express: la aplicación en sí, el objeto con la petición del cliente, y el objeto con la respuesta que se va a enviar.

La aplicación

La aplicación es una instancia de un objeto Express, que típicamente se asocia a una variable llamada **app** en el código:

```
const express = require('express');  
let app = express();
```

Toda la funcionalidad de la aplicación (métodos de respuesta a peticiones, inclusión de *middleware*, etc.) se asienta sobre este elemento. Cuenta con una serie de métodos útiles, que iremos viendo en futuros ejemplos, como son:

- » **use(middleware)**: para incorporar *middleware* al proyecto.
- » **set(propiedad, valor) / get(propiedad)**: para establecer y obtener determinadas propiedades relativas al proyecto.
- » **listen(puerto)**: para hacer que el servidor se quede escuchando por un puerto determinado.
- » **render(vista, [opciones], callback)**: para mostrar una determinada vista estática como respuesta, de forma que se puedan especificar opciones adicionales y un *callback* de respuesta.
- » ...

La aplicación

El objeto de petición (típicamente lo encontraremos en el código como **req**) se crea cuando un cliente envía una petición a un servidor Express. Contiene varios métodos y propiedades útiles para acceder a información contenida en la petición, como:

- » **params**: la colección de parámetros que se envía con la petición.
- » **query**: con la *query string* enviada en una petición GET.
- » **body**: con el cuerpo enviado en una petición POST.
- » **files**: con los archivos subidos desde un formulario en el cliente.
- » **get (cabecera)**: un método para obtener distintas cabeceras de la petición, a partir de su nombre.
- » **path**: para obtener la ruta o URI de la petición.
- » **url**: para obtener la URI junto con cualquier *query string* que haya a continuación.

La respuesta

El objeto respuesta se crea junto con el de la petición, y se completa desde el código del servidor Express con la información que se vaya a enviar al cliente. Típicamente se representa con la variable **res**, y cuenta, entre otros, con estos métodos y propiedades de utilidad:

- » **status (codigo)**: establece el código de estado de la respuesta.
- » **set (cabecera, valor)**: establece cada una de las cabeceras de respuesta que se necesiten.
- » **redirect (estado, url)**: redirige a otra URL, con el correspondiente código de estado.
- » **send ([estado], cuerpo)**: envía el contenido indicado, junto con el código de estado asociado (de forma opcional, si no se envía este por separado).
- » **json ([estado], cuerpo)**: envía contenido JSON específicamente, junto con el código de estado asociado (opcional).

Esquema 5.

3.2. Ejemplo de enrutamiento simple

En este apartado veremos cómo emplear un enrutamiento simple para ofrecer diferentes servicios utilizando Mongoose con una base de datos MongoDB. Para ello, crearemos una carpeta llamada **PruebaContactosExpress** en nuestra carpeta de pruebas (**ProyectosNode/Pruebas**). Instalaremos Express y Mongoose en ella, lo que puede hacerse con un simple comando; con esto, ya tendremos creado el archivo **package.json**:

```
npm install mongoose express
```

Nuestra aplicación tendrá un archivo **index.js**, donde incorporaremos tanto el modelo de datos de contactos (el mismo visto en sesiones previas) como las rutas para responder a diferentes operaciones sobre los contactos.

IMPORTANTE

Existe otra forma más adecuada de estructurar una aplicación Node/Express, especialmente cuando el número de colecciones u operaciones sobre estas es grande. Pero de momento vamos a optar por añadirlo todo a un mismo archivo para simplificar el código.

IMPORTANTE

Vamos a definir la estructura básica que va a tener nuestro servidor `index.js`, antes de añadirle las rutas para dar respuesta a los servicios. Esta estructura consistirá en incorporar Express y Mongoose, conectar con la base de datos y definir el modelo y esquema para los contactos.

Puedes consultar más información en el archivo Esquema en Mongoose e instalación y configuración de MongoDB, en la sección Descarga de documentos de la unidad.

```
/*
Pruebas con Express y Mongoose para proporcionar servicios básicos
sobre una base de datos de contactos
*/
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
// Conexión con la BD
mongoose.Promise = global.Promise;
mongoose.connect('mongodb://localhost:27017/contactos');
// Definición del esquema de nuestra colección
let contactoSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true,
    minlength: 1,
    trim: true
  },
  telefono: {
    type: String,
    required: true,
    unique: true,
```

```
      trim: true,
      match: /^\\d{9}$/
    },
    edad: {
      type: Number,
      min: 18,
      max: 120
    }
  });
// Asociación con el modelo (colección contactos)
let Contacto = mongoose.model('contacto', contactoSchema);
// Servidor Express
let app = express();
// Puesta en marcha del servidor
app.listen(8080);
```

A. Listado de todos los contactos

El servicio que lista todos los contactos es el más sencillo: atenderemos por GET a la URI `/contactos`, y en el código haremos un `find` de todos los contactos, y se devolverá directamente como resultado:

```
// Servicio de listado general
app.get('/contactos', (req, res) => {
  Contacto.find().then(resultado => {
    res.send(resultado);
  }).catch (error => {
    res.send([]);
  });
});
```

Observa que enviamos como respuesta el `array` de objetos obtenido, y es Express el que se encarga de transformarlo a JSON por nosotros. En caso de error, enviaremos un `array` vacío.

B. Ficha de un contacto a partir de su id

Veamos ahora cómo procesar con Express URI dinámicas. En este caso, accederemos por GET a una URI con el formato `/contactos/:id`, siendo `:id` el id del contacto que queremos obtener. Si especificamos la URI con ese mismo formato en Express, automáticamente se asocia al parámetro que venga a continuación de `/contactos` el nombre `id`, con lo que podemos acceder a él directamente por el objeto `req.params` de la petición. De este modo, el servicio queda así de simple:


```
// Servicio de listado por id
app.get('/contactos/:id', (req, res) => {
  Contacto.findById(req.params.id).then(resultado => {
    if(resultado)
      res.send({error: false, resultado: resultado});
    else
      res.send({error: true, mensajeError: "No se han encontrado contactos"});
  }).catch (error => {
    res.send({error: true, mensajeError: "Error al buscar el contacto indicado"});
  });
});
```

Hay que hacer notar que lo que devolvemos en la respuesta es un objeto JavaScript con tres campos:

- » **error**, que establece si ha habido algún error procesando la petición (**true**) o no (**false**).
- » **mensajeError**, que está presente solo si **error** es **true**. En ese caso, almacena el mensaje de error concreto que se ha producido.
- » **resultado**, que está presente si **error** es **false**. Contiene el resultado de la búsqueda.

Si la llamada a **findById** funciona correctamente, iremos a la sección **then**, y comprobaremos si hay un resultado válido. Si lo hay, establecemos **error=false** y, en caso contrario, haremos **error=true** con el correspondiente mensaje de error. Si algo falla y se lanza una excepción, iremos a la cláusula **catch** y enviaremos el correspondiente mensaje de error al cliente.

C. Inserción de contactos (POST)

Vamos a insertar un nuevo contacto, pasando en el cuerpo de la petición los datos de este (nombre, teléfono y edad) en formato JSON. En esta ocasión, vamos a valernos de un middleware para Express llamado **body-parser**, que facilita el procesamiento del cuerpo de las peticiones para acceder directamente a los datos que se envían en ella. En primer lugar, deberemos instalar el módulo en nuestro proyecto:

```
npm install body-parser
```

Después, lo incluimos con **require** junto al resto:

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
...
```

Y finalmente, lo añadimos como middleware con `app.use`, justo después de inicializar la app. Como lo que vamos a hacer es trabajar con objetos JSON, añadiremos el procesador JSON de `body-parser`:

```
// Servidor Express
let app = express();
// Body-parser para procesar datos JSON desde el cuerpo de las peticiones
// POST/PUT
app.use(bodyParser.json());
...
```

Ahora vamos a nuestro servicio POST. Observa qué sencillo queda empleando este *middleware*:

```
// Servicio para insertar contactos
app.post('/contactos', (req, res) => {
  let nuevoContacto = new Contacto({
    nombre: req.body.nombre,
    telefono: req.body.telefono,
    edad: req.body.edad
  });
  nuevoContacto.save().then(resultado => {
    res.send({error: false, resultado: resultado});
  }).catch(error => {
    res.send({error: true, mensajeError: "Error al añadir contacto"});
  });
});
```

D. Modificación de contactos (PUT)

La modificación de contactos es estructuralmente muy similar a la inserción: enviaremos en el cuerpo de la petición los datos nuevos del contacto que se va a modificar (a partir de su id), y utilizaremos `body-parser` para obtenerlos y llamar a los métodos apropiados de Mongoose para realizar la modificación del contacto. La URI a la que asociaremos este servicio será similar a la del POST, pero añadiendo el id del contacto que queramos modificar:

```
// Servicio para modificar contactos
```

```
app.put('/contactos/:id', (req, res) => {
  Contacto.findByIdAndUpdate(req.params.id, {
    $set: {
      nombre: req.body.nombre,
      telefono: req.body.telefono,
      edad: req.body.edad
    }
  }, {new: true}).then(resultado => {
    res.send({error: false, resultado: resultado});
  }).catch(error => {
    res.send({error: true, mensajeError: "Error al actualizar contacto"});
  });
});
```

Como se puede ver, obtenemos el id desde los parámetros (**req.params**) como cuando consultábamos la ficha de un contacto, y utilizamos dicho id para buscar al contacto en cuestión y actualizar sus campos con **findByIdAndUpdate**. En este punto, volvemos a hacer uso de la librería **body-parser** para procesar los datos que llegan desde el cuerpo de la petición. Tras la llamada al método, devolvemos un objeto JSON con los tres atributos que ya hemos visto en servicios anteriores (**error**, **mensajeError** y **resultado**).

04 Mecanismos de autenticación y verificación

En una **API REST** también puede ser necesario proteger ciertos servicios, de forma que solo puedan acceder a ellos los usuarios autenticados. Sin embargo, en este caso no tenemos disponible el mecanismo de autenticación basado en sesiones que vimos en temas anteriores, ya que la parte cliente que consulta la **API REST** no tiene por qué estar basada en un navegador. Podríamos acceder desde una aplicación de escritorio hecha en Java, por ejemplo, o desde una aplicación móvil, y en estos casos no podríamos disponer de las sesiones, propias de clientes web o navegadores. En su lugar, emplearemos un mecanismo de autenticación basado en *tokens*.

4.1. Fundamentos de la autenticación basada en *tokens*

La autenticación basada en *tokens* es un mecanismo de validación de usuarios en aplicaciones cliente-servidor que podríamos decir que es más universal que la autenticación basada en sesiones, ya que permite autenticar usuarios provenientes de distintos tipos de clientes. Lo que se hace es lo siguiente:

- » El usuario necesita enviar sus credenciales (**login y password**), de forma similar a como se hace en una aplicación web normal, aunque esta vez los datos se envían normalmente en formato JSON.
- » El servidor valida esas credenciales y, si son correctas, genera una cadena de texto llamada *token*, de una cierta longitud, y que servirá para identificar unívocamente al usuario a partir de ese momento. Dicho *token* debe ser enviado de vuelta (también en formato JSON) al cliente que se validó.
- » A partir de este punto, el cliente debe adjuntar el *token* como parte de la información en cada petición que realiza a una zona de acceso restringido, de forma que el servidor pueda consultar el *token* y comprobar si se corresponde con el de algún usuario autorizado. Este *token* normalmente se envía en una cabecera de la petición llamada **Authorization**, como veremos después, y el servidor puede consultar el valor de dicha cabecera para verificar el acceso del cliente.

4.2. Implementación de la autenticación basada en *tokens* con Laravel

Podemos emplear distintas alternativas para la autenticación basada en *tokens* bajo Laravel. Comentaremos en este apartado dos de ellas.

Por un lado, podemos emplear el mecanismo nativo de Laravel para autenticación basada en *tokens*. Como ventajas principales, no se necesita instalar ninguna dependencia adicional, y es relativamente sencillo de utilizar. Como inconvenientes, requiere añadir un campo más a la tabla de usuarios, para almacenar el *token* generado para cada usuario, y requiere también de una gestión manual del *token*, aunque es sencilla.

Por otro lado, podemos valernos de la librería **Laravel Sanctum**, que proporciona mecanismos de autenticación para API y para SPA (***single-page applications***, aplicaciones de página única). Entre sus ventajas podemos destacar que es sencilla de integrar en la aplicación y automatiza algunos aspectos de la gestión de *tokens*, además de contar con el soporte oficial de Laravel. Como inconvenientes, es una librería más intrusiva que la anterior, ya que requiere crear una tabla adicional donde almacenar los *tokens*.

Puedes encontrar más información sobre la autenticación usando Laravel Sanctum y autenticación en Postman en el documento Mecanismos de autenticación y verificación, en la sección Descarga de documentos de la unidad.

En los siguientes subapartados veremos cómo proteger mediante *tokens* un proyecto sencillo en Laravel empleando cada uno de estos mecanismos.

A. Preparar el ejemplo base

Partiremos de un mismo proyecto base, que luego adaptaremos en función del mecanismo de autenticación elegido. Comenzaremos creando un proyecto llamado `testToken`, en nuestra carpeta de proyectos:

```
laravel new testToken
o
composer create-project laravel/laravel testToken
```

Después, eliminaremos las migraciones que no vamos a utilizar de la carpeta `database/migrations`: en concreto, eliminaremos los archivos sobre `create_password_resets_table` y `create_failed_jobs_table`, y dejaremos solo la migración de la tabla de usuarios. Sobre ella, editaremos los métodos `up` y `down` para dejar solo los campos que nos interesen:

```
public function up()
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}

public function down()
{
    Schema::dropIfExists('users');
}
```

A continuación, vamos al modelo `App\Models\User.php`:

```
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
```

```

protected $fillable = [
    'name' ,
    'email' ,
    'password' ,
];

protected $hidden = [
    'password' ,
    'remember_token' ,
];

protected $casts = [
    'email_verified_at' => 'datetime' ,
];
}

```

Y hacemos lo mismo con el **factory** y el **seeder** correspondiente:

```

class UsuarioFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition()
    {
        return [
            'name' => $this->faker->name() ,
            'email' => $this->faker->unique()->safeEmail() ,
            'email_verified_at' => now() ,
            'password' => '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi' , // password
            'remember_token' => Str::random(10) ,
        ];
    }
}

class DatabaseSeeder extends Seeder
{
    public function run()
    {
        \App\Models\User::factory(10)->create();
    }
}

```

Vamos a modificar también el archivo `.env` del proyecto para acceder a una base de datos llamada `testToken`, que deberemos crear a través de phpMyAdmin:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=testtoken
DB_USERNAME=root
DB_PASSWORD=
```

Necesitamos también editar el archivo `App\Http\Middleware\Authenticate.php` para indicar en el método `redirectTo`, que solo queremos redirigir al formulario de login cuando la petición no espere una respuesta en formato JSON. En caso contrario, no hay que mostrar dicho formulario, sino enviar una respuesta JSON adecuada. De hecho, si la aplicación solo va a tener servicios **REST**, podríamos eliminar o dejar comentado el código de este método para que no trate de redirigir a ningún formulario.

```
class Authenticate extends Middleware
{
    protected function redirectTo($request)
    {
        /* if (! $request->expectsJson()) {
            return route('login');
        } */
    }
}
```

Por otra parte, debemos editar el archivo `App\Exceptions\Handler.php` (en concreto, su método `register`) para definir los diferentes errores que pueden producirse y los mensajes que hay que devolver en cada caso:

```
public function register()
{
    $this->renderable(function (Throwable $exception) {
        if (request()->is('api*')) {
            if ($exception instanceof ModelNotFoundException)
                return response()->json(['error' => 'Elemento no encontrado'], 404);
            else if ($exception instanceof AuthenticationException)
                return response()->json(['error' => 'Usuario no autenticado'], 401);
            else if ($exception instanceof ValidationException)
                return response()->json(['error' => 'Datos no válidos'], 400);
        }
    });
}
```

```

        else if (isset($exception))
            return response()->json(['error' => 'Error en la aplicación (' .
get_class($exception) . '):' . $exception->getMessage()], 500);
    }
    });
}

```

Finalmente, vamos a definir un controlador de API con una serie de métodos de prueba. No lo vamos a vincular a ningún modelo, porque generaremos unos datos a mano en cada método para simplificar el código. Escribimos este comando:

```
php artisan make:controller Api/TestController --api
```

Rellenamos el código de los métodos del controlador con alguna respuesta sencilla para cada caso:

```

class TestController extends Controller
{
    public function index()
    {
        return response()->json(['mensaje' => 'Accediendo a index']);
    }

    public function store(Request $request)
    {
        return response()->json(['mensaje' => 'Insertando']);
    }

    public function show($id)
    {
        return response()->json(['mensaje' => "Mostrando Ficha[$id]"]);
    }

    public function update(Request $request, $id)
    {
        return response()->json(['mensaje' => "Actualizando elemento [$id]"]);
    }

    public function destroy($id)
    {
        return response()->json(['mensaje' => "Borrando elemento [$id]"]);
    }
}

```

Y añadimos las rutas correspondientes en el archivo `routes/api.php`:

```
Route::apiResource('prueba', PruebaController::class);
```


A partir de este punto, vamos a proteger el acceso a alguno de estos métodos. Escoge uno de los siguientes subapartados (4.3 o 4.4) para definir el mecanismo de autenticación basado en *tokens* correspondiente. También puedes intentar hacerlos todos; en ese caso, copia y pega otra vez el proyecto Laravel, para trabajar por separado en cada carpeta con un mecanismo diferente.

4.3. Autenticación basada en *tokens* nativa con Laravel

Vamos a emplear en esta sección la autenticación nativa por *tokens* que ofrece Laravel. Los pasos que hay que seguir los indicamos a continuación.

A. Configuración básica

En primer lugar, modificamos la migración de la tabla de usuarios para añadir un nuevo campo donde almacenar el *token*. Basta con que dicho campo tenga 60 caracteres de longitud, y será necesario también que sea único para cada usuario:

```
public function up()
{
    Schema::create('usuarios', function (Blueprint $table) {
        $table->id();
        $table->string('login')->unique();
        $table->string('password');
        $table->string('api_token', 60)->unique()->nullable();
        $table->timestamps();
    });
}
```

Podemos lanzar ya la migración para que se cree la tabla y se rellene con los usuarios que hayamos indicado en el *seeder*.

```
php artisan migrate:fresh --seed
```

También debemos modificar el archivo `config/auth.php` para indicar cuál es el modelo de usuarios que vamos a utilizar:

```
'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => App\Models\Usuario::class,
    ],
],
```

En este mismo fichero, también podemos modificar el **guard** por defecto, que es **web**, para que sea **api**, si nuestra aplicación no va a tener autenticación web:

```
'defaults' => [  
  'guard' => 'api',  
  'passwords' => 'users',  
],
```

B. Protección de rutas

Para proteger las rutas de acceso restringido, primero crearemos un controlador que se encargue de validar las credenciales del usuario:

```
php artisan make:controller Api/LoginController
```

Definimos un método **login**, por ejemplo, que validará las credenciales que le lleguen (**login** y **password**). Si son correctas, generará una cadena de texto aleatoria de 60 caracteres y la almacenará en el campo **api_token** del usuario validado. También devolverá dicho token como respuesta en formato JSON. En caso de que haya un error en la autenticación, enviará de vuelta un mensaje de error, con el código 401 de acceso no autorizado.

```
class LoginController extends Controller  
{  
    public function login(Request $request)  
    {  
        $usuario = Usuario::where('login', $request->login)->first();  
        if (!$usuario || !Hash::check($request->password, $usuario->password))  
        {  
            return response()->json(['error' => 'Credenciales no válidas'], 401);  
        }  
        else  
        {  
            $usuario->api_token = Str::random(60);  
            $usuario->save();  
            return response()->json(['token' => $usuario->api_token]);  
        }  
    }  
}
```

Definimos en el archivo `routes/api.php` una ruta que redirija a este método, para cuando el usuario quiera autenticarse (recuerda añadir con `use` la correspondiente clase):

```
Route::post('login', [LoginController::class, 'login']);
```

También podemos eliminar en este caso la ruta predefinida de este archivo, que emplea la autenticación nativa de Laravel:

```
// Eliminar esta ruta:  
Route::middleware('auth:api')->get('/user', function (Request $request)  
{  
    return $request->user();  
});
```

Para proteger las rutas que necesitemos en los controladores API, las especificamos en el constructor del controlador. Por ejemplo, así protegeríamos todas las rutas de nuestro controlador `PruebaController`, salvo `index` y `show`, como indica el modificador `except`:

```
class PruebaController extends Controller  
{  
    public function __construct()  
    {  
        $this->middleware('auth:api', ['except' => ['index', 'show']]);  
    }  
}
```

Como alternativa, también podemos emplear el modificador `only` en lugar de `except` para indicar las rutas concretas que queremos proteger; es decir, con `only` le decimos las rutas que queremos proteger y con `except` las que no queremos proteger.



VÍDEOLAB

Para consultar el siguiente vídeo sobre página de productos con SOAP, escanea el código QR o [pulsa aquí](#).

UAX FP